

証明項を明かさないう定理検証 Lean 4 kernel の zkVM 実装

1. Lean 4 と、証明項を隠す目的
2. zkVM + Lean 4 の設計
3. 証明・検証のフロー
4. 大きな prelude の渡し方
5. Mathlib 上での評価

Lean 4 とは

Lean 4 は依存型に基づく定理証明支援系。命題を型、証明をその型を持つ項として表し、項を検査するだけで定理の成立を確かめられる。

statement

定理そのもの。Lean では型として表される。
公開するのはこの型の digest で、証明の中身は含まない。

proof term

証明の中身そのもの。tactic で組み立てる。
export すると tactic の痕跡は消え、この項だけが残る。

kernel

項の型が statement と一致するかを検査する。
Lean が信用しているのはここで、通れば定理が成り立つとみなす。

prelude

証明を読むために先に入れておく宣言の集まり。
今回は Mathlib 由来の 214,101 宣言を固定して使う。

証明を組み立てる elaborator や tactic は、zkVM の外に置いている。

zkVM に載せる目的：証明項を隠す

statement (公開)

theorem thm_factor : $\exists p\ q : \mathbb{N}, p.\text{Prime} \wedge q.\text{Prime} \wedge p \neq q \wedge p * q = 3233 :=$

$\langle 61, 53, \text{by norm num}, \text{by norm num}, \text{by norm num}, \text{by norm num} \rangle$

proof term (秘密)

証明項は 61 と 53 を含む

61 と 53 が現れるのは項の中だけで、statement には出てこない。

statement に因数の値は直接書かない

3233 が相異なる 2 つの素数の積である、という statement は、その素数を知らなくても書ける。

渡すのは型検査が通った事実

項を見せずに、kernel が受け付けたことだけを検証者に確かめてもらう。

zkVM + Lean 4 の設計

ホスト側 elaborator・tactic で証明を組み立て、proof term を export する
zkVM image Lean kernel の独立した Rust 実装 (nanoda_lib) が proof term を検査し直す

	公開	秘密
public prelude (検証者も同じものを持っている)	○	
statement の canonical digest	○	
allowlist 外の axiom が無いこと	○	
proof term と、補題をどう並べたか		○

prelude は検査せず root で出所を縛り、proof term は出所を問わず全部検査する。

証明項を隠すため、公理への不正な依存も検査する。許可外の公理を拒否し、公開公理の型を監査する。

証明・検証のフロー

Lean4 export は public = prelude と private = proof term に割る。

レコードは prelude を項ごとに分解した DAG のノード。



検証者が確かめること

image ID

公開されているものと同じか

prelude

自分の public.bin と同じか

statement

指定の 1 件と digest が合うか

axiom

allowlist 外も偽物も無いか

公開 prelude は検証者が事前に型検査・公理監査する。receipt が保証するのは、その環境に対する型検査。

image ID はプログラムの中身から決まるダイジェスト。本物かは Docker の再現ビルドで確かめる。

大規模 prelude の安全な部分読み込み

公開 prelude は 682 MB ・ 3,728 万レコード。全件を毎回 zkVM guest に入力するのは実用的でない。
レコード = export の 1 行。各レコードは整数 ID で他のレコードを参照する。

	公開環境との 一致	実用性
必要なレコードだけ渡す（認証なし） 公開レコードを偽物に差し替えられる	—	○
prelude 全体を渡し、digest を照合 682 MB を毎回 guest に入力する	○	—
必要なレコード + Merkle proof 省略した部分木の root から全体 root を再構成する	○	○

受け取ったレコードが固定済みの公開環境に属することを、guest が確認できる。

実測: 公開環境の宣言数を 168 倍にしても、zkVM cycles は 1.08 倍にとどまった (62.97M → 67.71M) 。

Mathlib 上の 5 定理での評価

いずれも同じ公開 prelude (214,101 宣言)	cycles	GPU 証明生成	receipt 検証
素数は無限にある (Euclid)	1.76M	6.8 s	0.07 s
$\sqrt{2}$ は無理数	2.62M	10.1 s	0.07 s
Gauss の和公式 $0+1+\cdots+(n-1)$	3.66M	12.9 s	0.07 s
3233 は相異なる 2 素数の積	30.55M	95.2 s	0.06 s
相加相乗平均 $2ab \leq a^2+b^2$	61.96M	197.2 s	0.06 s

固定の大きな公開環境に対して、証明項を検証者に公開せずに定理を検証できた。

61 と 53 を含む proof term は private input のまま、receipt からは公開されない。

時間は GPU 証明生成と receipt 検証のみ。export と公開環境の準備・監査は含まない。